# *Regular Languages and Regular Expressions*

Birzhan Kalmurzayev

Ac. year 2025-2026

# *Strings and Languages*

*Definition*

A **string** (or **word**) is a finite sequence of symbols

In formal theory, it is necessary to fix the set of symbols used to form strings. Such finite set of symbols is called an **Alphabet**.

For example:

- $\{a, b, c, d, e, \ldots x, y, z\}$ is Roman alphabet
- $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ is Arabic digits
- $\{0, 1\}$ is binary alphabet

A string over the binary alphabet is called a **binary string**.

The **length** of a string $\omega$, denoted by $|\omega|$, is the number of symbols contained in the string.

For example:

- $|mathematics| = 11$;
- $|1001| = 4$;

The **empty string**, denoted by $\varepsilon$, is a string having no symbol. Clearly, $|\varepsilon| = 0$.

Let $x$ and $y$ be two string, and assume $x = x_1x_2x_3 \cdots x_n$, $y = y_1y_2y_3 \cdots y_m$ where each $x_i$ and $y_i$ are symbols. Then, $x$ and $y$ are equal iff $n = m$ and $x_i = y_i$ for all $i = 1, 2, 3, \ldots, n$.

For example: $1010 \neq 0101$

# *Concatenation*

*Definition*

The **concatenation** $x \cdot y$ of two strings $x$ and $y$ is the string $xy$, that is, $x$ followed by $y$.

For example: *spidermen* is the concatenation of *spider* and *men*.

For any string $x$ and non-negative integer $k$, by $x^k$ we denote the string which obtained by $x$ using $k$-times concatenation to itself. That is $x^0 = \varepsilon$, $x^1 = x$, $x^2 = x \cdot x$, ...

For example: if $x = 001$, then $x^5 = 001001001001001$.

Let $x$ be a string. A string $s$ is a **substring** of $x$ if there exists strings $y$ and $z$ such that $x = y \cdot s \cdot z$. In particular,

- when $x = s \cdot z$, then $s$ is called a **prefix** of $x$,
- when $x = y \cdot s$ then $s$ is called **suffix** of $x$.

For a string $x$, the **reversal** of $x$, denoted by $x^R$, is defined by

$$x^R = \begin{cases} \varepsilon, & \text{if } x = \varepsilon \\ x_n x_{n-1} \cdots x_2 x_1, & \text{if } x = x_1 x_2 \cdots x_{n-1} x_n \end{cases}$$

where each $x_i$ is a symbol.

For example: $mathematics^R = scitamehtam$

# *Languages*

A **language** is a set of strings (words). For example, the set of all English words is language over the Roman alphabet. If $\Sigma$ is an alphabet, then by $\Sigma^*$ we denote the set of all strings over $\Sigma$. Thus, a language $L$ over the alphabet $\Sigma$ is just a subset of $\Sigma^*$.

Languages are set, so the operations as **intersection**, **union** and **subtraction** are defined as on sets.

- **Complementation**: If $A$ is a language over the alphabet $\Sigma$, then $\overline{A} = \Sigma^* \setminus A$.
- **Concatenation**: If $A$ and $B$ are two languages, then their concatenation is $A \cdot B = \{ab : \ a \in A \text{ and } b \in B\}$.

For example: If $A = \{0, 1\}$ and $B = \{1, 2, 3\}$, then $AB = \{01, 02, 03, 11, 12, 13\}$

# *Kleene star*

For any language $A$, we define
$A^0 = \{\varepsilon\}$,
$A^1 = A$,
$A^2 = A \cdot A$,
$A^3 = A^2 \cdot A$, ...

**Kleene closure** (or **star closure**): For any language $A$, define

$$A^* = A^0 \cup A^1 \cup A^2 \cup A^3 \cup \cdots =$$

$$= \{\omega : \ \omega \text{ is the concatenation of 0 or more string from } A\}$$

**Example**: Describe the language $\{0, 10\}^*$

# *Examples*

Prove the following equalities for languages:

- $(AB)^R = B^R A^R$;
- $(A \cup B)^R = A^R \cup B^R$;
- $(A \cup B)^* = A^*(BA^*)^*$.

# *Regular languages*

*Definition*

The **regular languages** over an alphabet $\Sigma$ is defined recursively as follows:

1. The empty set $\emptyset$ is a regular language;
2. For every $a \in \Sigma$, $\{a\}$ is a regular language;
3. If $A$ and $B$ are regular languages, then
   - $A \cup B$;
   - $AB$;
   - $A^*$

   are all regular languages.

Example: The language $A = \{001, 101\}$ is regular.

# *Regular expression*

*Definition*

To simplify the representations for regular languages, we define the notion of **regular expressions** over an alphabet $\Sigma$ as follows:

1. $\emptyset$ is a regular expression which represents the empty set;
2. $\varepsilon$ is a regular expression which represents the language $\{\varepsilon\}$;
3. For $a \in \Sigma$, $a$ is a regular expression which represents language $\{a\}$;
4. If $r_A$ and $r_B$ are regular expressions which representing languages $A$ and $B$, respectively, then
   - $(r_A) + (r_B)$ is regular expression representing language $A \cup B$;
   - $(r_A)(r_B)$ is regular expression representing language $AB$;
   - $(r_A)^*$ is regular expression representing language $A^*$.

For any regular expression $r$, we let $L(r)$ denote the regular language represented by $r$.

To make it simplify to write regular expressions, we will set the following priorities:

1. Kleene closure has the higher priority over plus and concatenation;
2. Concatenation has the higher priority over plus.

For example: The regular expression

$$(0^* + 100^*)10^*(01^* + 1^*)$$

is the regular expression

$$(((0)^*) + ((1)(0))(0)^*)(1)((0)^*)(((0)(1)^*) + (1^*))$$

Assume

$$r = (0^* + 100^*)10^*(01^* + 1^*)$$

Determine whether the following strings belong or do not belong to the language $L(r)$

- $\varepsilon$;
- 1;
- 01001011;
- 11010010.

# *Properties of regular expressions*

For any regular expressions $r$, $s$ and $t$

- $r(s+t) = rs + rt$;
- $(r+s)t = rt + st$;
- if $r \subseteq s$, then $r + s = s$

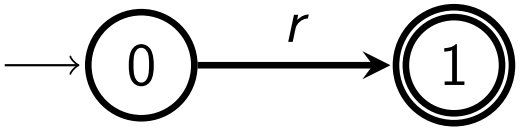
**Examples:** Find regular expressions for the following languages:

- $A$ is set of binary expressions of integers which are the power of 4;
- $B$ is set of binary strings which have at least one occurrence of the substring 001;
- $C$ is set of binary strings which have no substring 001

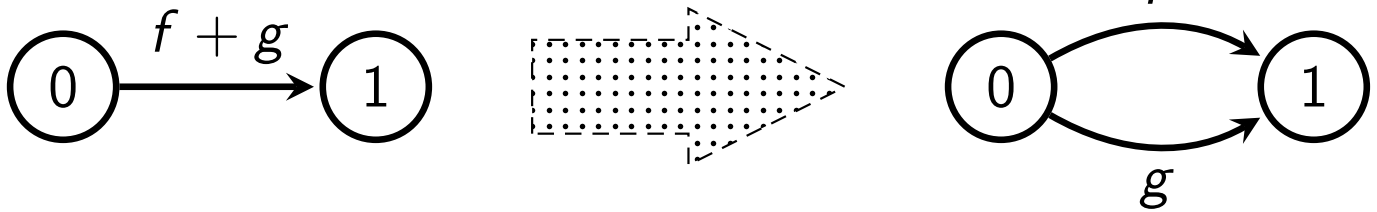
For any regular expression $r$, its labeled digraph representation can be obtained as follows:

1. Initially, we start with two special vertices, the initial vertex and the final vertex, and draw an egde between them with label $r$ as in figure in the next slide (1).
2. Repeat the following until every edge has a label that does not contain operation symbols $+, \cdot$ and $*$.
   - Replace edge with label $f + g$ by two edges with labels $f$ and $g$ as shown in the next slide;
   - Replace edge with label $fg$ by addition new vertex and two edges with labels $f$ and $g$ as shown in the next slide;
   - Replace edge with label $f^*$ by addition new vertex and three edges with labels $\varepsilon$ and $f$ as shown in the next slide;
3. Delete all edges with label $\emptyset$

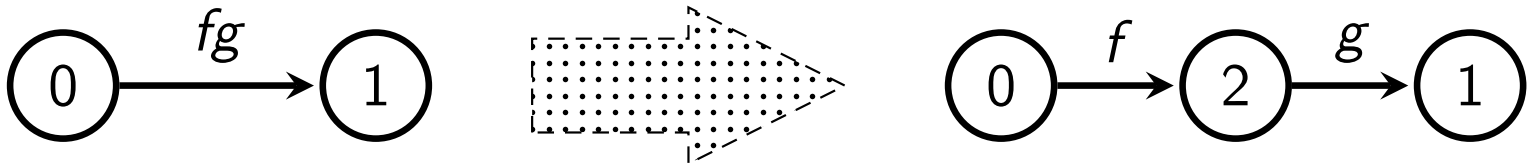
(1)



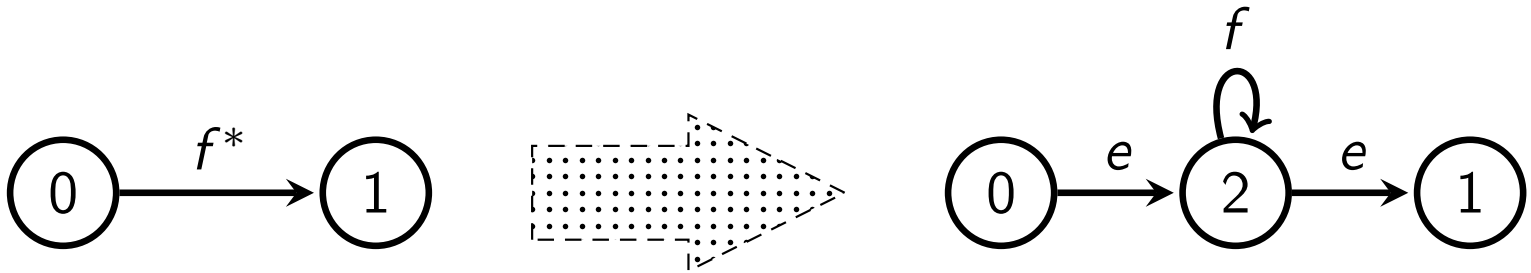

(2)




(3)




(4)

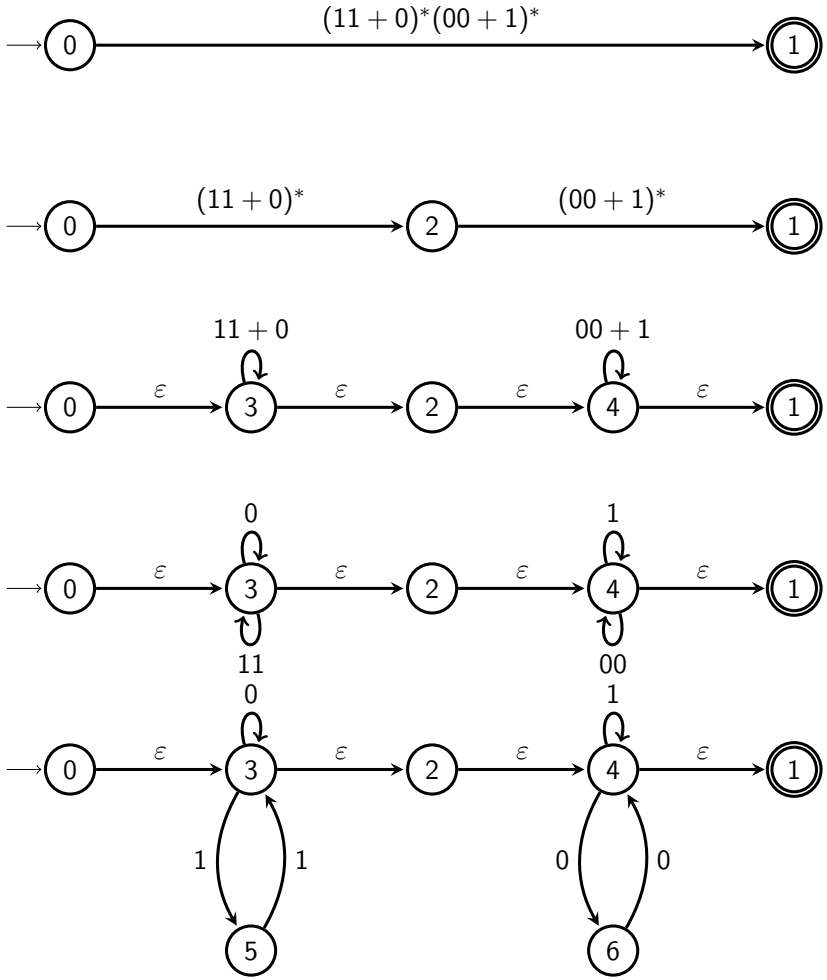


Every path in the obtained graph (lets denote it as $G(r)$) is associated with a string which obtained by concatenating all symbols labeling the edges in the path.

*Theorem*

*Let $r$ be a regular expression. A string $x$ belong the the language $L(r)$ if and only if there is a path in $G(r)$ from the initial vertex to the final vertex whose associated string is $x$*

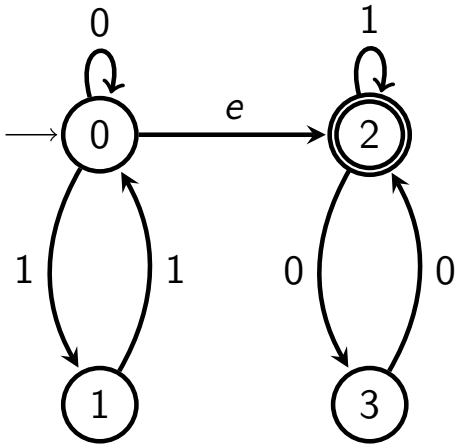
Example: Construct graph representation for the regular expression $r = (11 + 0)^*(00 + 1)^*$



*Theorem*

*Let $r$ be a regular expression. Then $\varepsilon$-edge $(u, v)$ in $G(r)$ which is a unique out-edge from a non-final vertex $u$ or a unique in-edge to a non-initial vertex $v$ can be shrunk into a single vertex, still preserving the property of previous theorem.*

The simpler solution for previous example:



# *Examples*

Construct $G(r)$ for $r = a^*b(c + da^*b)^*$